

Bachelor Thesis

Understanding Asynchronous Inference Methods for Vision-Language-Action Models

Ayoub Agouzoul^{1,2}

¹École polytechnique
²ETH Zürich

Supervised by Professor Wenqi Jiang

January—March 2026



Outline

- 1 Motivation
- 2 Background
- 3 Experimental Setup
- 4 Results
- 5 Conclusion



Outline

- 1 Motivation
- 2 Background
- 3 Experimental Setup
- 4 Results
- 5 Conclusion



Vision-Language-Action (VLA) models:

- Map camera images + language instructions $\rightarrow$ robot actions
- Leverage pretrained vision-language representations
- State-of-the-art: π_0 , $\pi_{0.5}$, SmolVLA

Action chunking for efficient deployment:

- Predict H future actions in one forward pass (via flow matching)
- Execute chunk while computing the next one (asynchronous)

Key problem

Large model $\rightarrow$ high inference latency $\rightarrow$ by the time actions are ready, the robot's state has changed

Asynchronous Inference Creates Observation Staleness

Inference delay $d = \lfloor \delta / \Delta t \rfloor$:

- δ : wall-clock inference time
- Δt : control period (e.g., 20 ms at 50 Hz)
- First d actions in each new chunk are **already obsolete**

Naive async execution causes:

- **Chunk boundary discontinuity** — abrupt transitions, jitter
- **Observation staleness** — actions conditioned on outdated state
- **Open-loop degradation** — no correction during chunk lifetime

How severe?

Cloud inference over 4G: $d = 13$ steps; edge GPU on-device: $d = 2$

Four Methods Address Staleness — Which Works Best?

- ① **IT-RTC**: Inpaints fresh actions at inference time (training-free)
- ② **TT-RTC**: Simulates delays during training (zero inference overhead)
- ③ **VLASH**: Conditions on estimated future robot state
- ④ **A2C2**: Lightweight correction head for per-step residual fixes

Problem: Developed independently, incompatible codebases and protocols

Our contribution

- **Two unified codebases** for fair side-by-side comparison
- **Systematic evaluation** on Kinetix and LIBERO (delays up to $d = 20$)
- **Cost analysis and practical guidelines** for choosing a method

Outline

- 1 Motivation
- 2 Background
- 3 Experimental Setup
- 4 Results
- 5 Conclusion



How Do VLA Models Generate Actions?

Architecture (e.g., SmolVLA, π_0):

- **VLM backbone:** Processes images + language $\rightarrow$ features
- **Action expert:** Generates continuous action chunk via flow matching

Flow matching for action generation:

- Start from noise $\epsilon \sim \mathcal{N}(0, I)$
- Iteratively denoise through n Euler steps (typically 5–10)
- $A^{\tau+1/n} = A^\tau + \frac{1}{n} v_\theta(A^\tau, o, \tau)$

Action chunking:

- Predict H future actions: $A_t = (a_t, \dots, a_{t+H-1})$
- Execute $s \leq H$ actions, then replan with fresh observation



IT-RTC and TT-RTC Fix Chunk Boundary Discontinuity

IT-RTC (inference-time):

- Treats committed prefix as constraint
- Soft mask + pseudoinverse VJP guidance during denoising
- **No retraining needed**
- Overhead: $3\times$ FLOPs on denoising

TT-RTC (training-time):

- Simulates random delays $d \sim p(d)$ during training
- Per-token flow timesteps: prefix at $\tau=1$, postfix sampled
- Loss computed only on postfix
- **Zero inference overhead**

Key difference

IT-RTC approximates via linearized VJP guidance; TT-RTC learns the prefix-conditioned distribution directly

VLASH and A2C2 Target Different Failure Modes

VLASH (future-state-aware):

- Estimates robot state at execution time:
 $\hat{s}_{t+\Delta} = s_t + \sum_k a_{t+k}^{\text{prev}}$
- Conditions policy on future state
- Temporal-offset augmentation during training
- Needs dynamics model or simulator

A2C2 (residual correction):

- Lightweight correction head runs every control step
- $a^{\text{exec}} = a^{\text{base}} + \Delta a$
- Uses *latest* observation
- **Plug-in:** no base policy retraining
- Can combine with other methods

Each method targets a different failure mode

IT-RTC & TT-RTC → chunk discontinuity, VLASH → staleness, A2C2 → open-loop degradation

Outline

- 1 Motivation
- 2 Background
- 3 Experimental Setup**
- 4 Results
- 5 Conclusion



Unified Codebases Enable Fair Comparison

Problem: Each method in separate codebase with different dependencies

bt-kinetix (JAX):

- All 4 methods + naive baseline
- MLP-Mixer backbone ($\sim 310\text{K}$ params)
- Chunk sizes $H \in \{16, 30\}$
- 2048 parallel rollouts per config

bt-libero (PyTorch / LeRobot):

- SmolVLA base policy (604M params)
- All 4 methods + naive baseline
- Unified evaluation harness
- 4 LIBERO task suites (40 tasks)

Key benefit

Same base policy, data, and evaluation protocol $\rightarrow$ performance differences attributable to method, not implementation

Two Benchmarks Cover Different Scales and Tasks

Kinetix (2D physics):

- 10 dynamic environments
- Throwing, catching, balancing, locomotion
- Symbolic observations ($\text{dim} = 679$)
- Fast JAX simulation on GPU
- Delays $d = 0, \dots, 15$

LIBERO (manipulation):

- 40 language-conditioned tasks
- 7-DoF Franka Panda arm
- RGB images + proprioception
- SmolVLA (production-scale VLA)
- Delays $d = 0, \dots, 20$

Research questions:

RQ1 How do methods compare as delay d increases?

RQ2 Do training-based methods generalize beyond $d_{\max}$?

RQ3 What are the training and inference costs of each method?



Real-World Delays Motivate Extended Evaluation Range

Scenario	Configuration	Latency δ	Delay d
Consumer GPU (local)	RTX 4090	31 ms	1
Edge GPU (on-device)	Jetson Thor	53 ms	2
Edge server (4G)	B100 + 4G	73 ms	3
Cloud server (slow)	B100 + 4G + cloud	273 ms	13

Based on π_0 latency estimates (Jiang et al.), 50 Hz controller

Prior work only evaluated $d \leq 4$

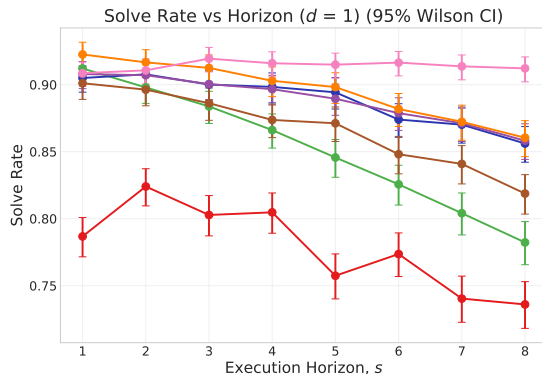
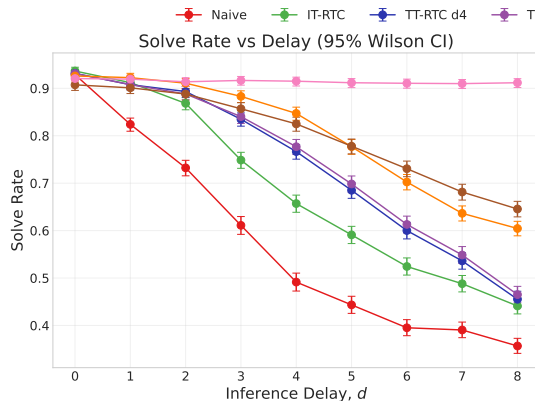
We extend to $d = 15$ (Kinetix) and $d = 20$ (LIBERO) to cover realistic deployment scenarios

Outline

- 1 Motivation
- 2 Background
- 3 Experimental Setup
- 4 Results**
- 5 Conclusion

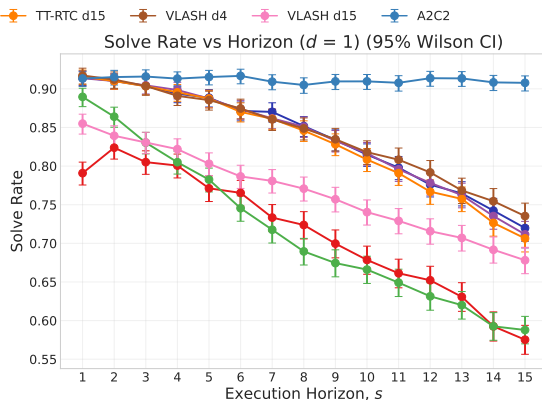
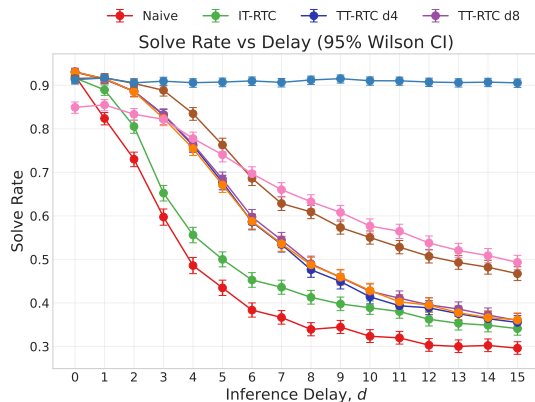


A2C2 Leads; TT-RTC Surpasses IT-RTC ($H = 16$)



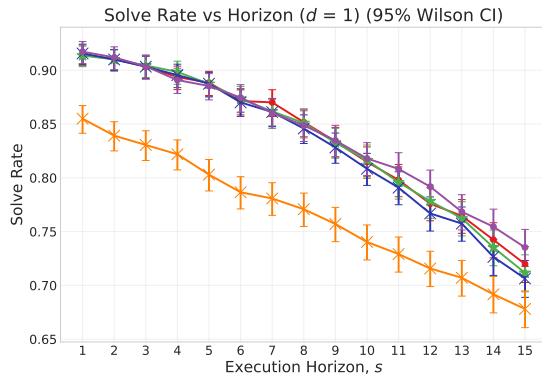
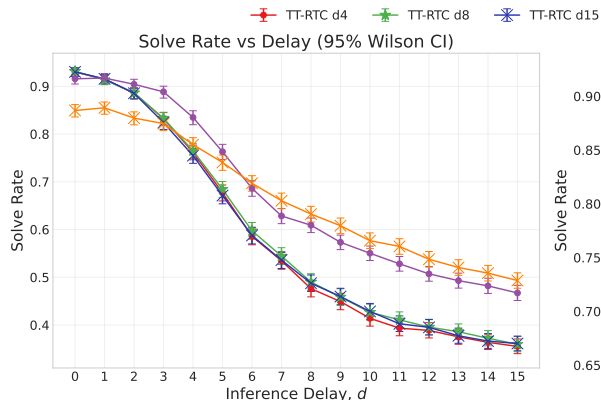
Kinetix, 10 envs, 2048 rollouts each. A2C2 (pink) leads, above 90% even at $d = 8$. VLASH outperforms RTC methods. TT-RTC $d_{\max}=8$ surpasses IT-RTC at higher delays. Naive (red) drops below 40%.

IT-RTC Degrades at High Delays ($H = 30$)



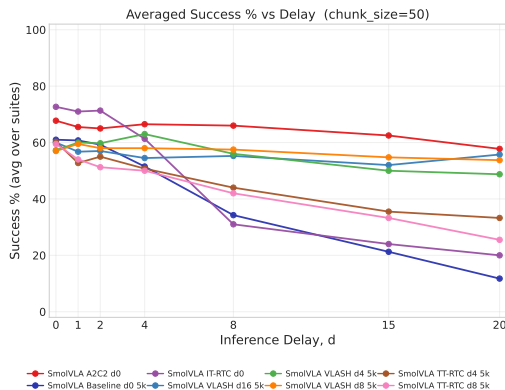
Extended to $d = 15$. A2C2 still leads. **IT-RTC falls close to naive** — VJP approximation degrades as prefix-to-postfix ratio grows. TT-RTC remains robust. VLASH $d_{\max}=15$ hurts low-delay performance ($\sim 10\%$ drop).

TT-RTC Generalizes Stably; VLASH Trades Low for High Delay



$d_{\max}$ **ablation** ($H = 30$). TT-RTC variants evolve very similarly regardless of $d_{\max}$. VLASH shows a clear **low-delay vs. high-delay trade-off**: wider $d_{\max}$ helps at high d but hurts near $d = 0$.

Results Extend to Production VLA Models on LIBERO



SmolVLA (604M params) on 40 LIBERO tasks, $s = 50$, delays $d \in \{0, \dots, 20\}$

VLA-Scale Cost Analysis Reveals Surprising Trade-offs

Method	Per-chunk GFLOPs	Ratio
Naive / TT-RTC / VLASH	551.0	1.00×
IT-RTC	663.8	1.20×
A2C2 ($s=50$)	961.7	1.75×

Why IT-RTC is only 1.2× (not 3×)?

- Vision encoder prefill = **89.8%** of chunk cost (494.6 / 551.0 GFLOPs)
- IT-RTC's 3× overhead applies only to the denoising loop (10% of cost)

Why A2C2 is the most expensive?

- 50 ResidualTransformer corrections $\times$ 8.21 GFLOPs each = 410.7 GFLOPs

Latency still matters

IT-RTC triples the *sequential* denoising bottleneck $\rightarrow$ avoid for real-time

Outline

- 1 Motivation
- 2 Background
- 3 Experimental Setup
- 4 Results
- 5 Conclusion**



Summary: No Single Winner, But Clear Trade-offs

Key findings:

- All four methods substantially outperform naive async execution
- A2C2's per-step correction most effective ($>90\%$ at $d=0\dots8$ on Kinetix)
- IT-RTC degrades at high delays and long chunks ($H = 30$)
- TT-RTC is the most **robust**: stable across $d_{\max}$, zero overhead
- VLASH has a clear low-delay vs. high-delay trade-off with $d_{\max}$

Practical guidelines:

- No retraining possible? $\rightarrow$ **IT-RTC** (drop-in, but degrades at high d)
- Fine-tuning feasible? $\rightarrow$ **TT-RTC** (best robustness–cost trade-off)
- Dynamics model available? $\rightarrow$ **VLASH** (tune $d_{\max}$ carefully)
- Want per-step corrections? $\rightarrow$ **A2C2** (plug-in, combinable)



- VLASH on LIBERO uses ground-truth future states (simulator), not available in real deployment
- Hyperparameters from original papers, not tuned per benchmark
- Inference delay d fixed per episode; real deployments have variable latency
- Limited GPU budget: not all method combinations and chunk sizes explored
- Walltime measurements on Kinetix not representative of real-world robot deployment

Questions and Discussion

Main takeaways:

- 1 Observation staleness is a critical VLA deployment bottleneck
- 2 No single method wins everywhere — the right choice depends on deployment constraints
- 3 Unified evaluation reveals trade-offs invisible when methods are compared separately

Code available

<https://github.com/TheAynos/async-vla-inference>

Acknowledgments

Prof. Wenqi Jiang, Prof. Sergio Mover, ETH Systems Group, École polytechnique

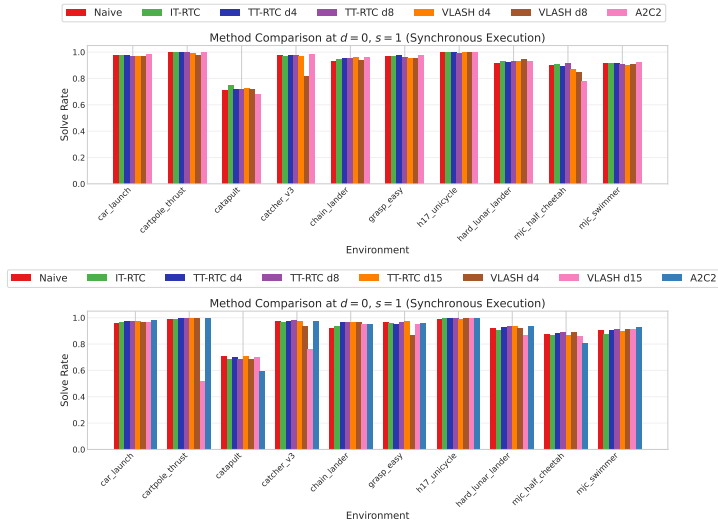
Appendix: Deployment Scenario Delay Mapping

Scenario	Configuration	Latency δ	Delay d
Datacenter GPU (local)	B100, no network	3.2 ms	0
Consumer GPU (local)	RTX 4090, no network	31.1 ms	1
Edge GPU (on-device)	Jetson Thor, no network	52.6 ms	2
Edge server (WiFi 7)	B100 + WiFi 7	8.4 ms	0
Edge server (5G)	B100 + 5G	27.8 ms	1
Edge server (4G)	B100 + 4G	73.0 ms	3
Cloud server (fast)	B100 + wired + fast cloud	23.4 ms	1
Cloud server (slow)	B100 + 4G + slow cloud	273.4 ms	13

Based on π_0 latency estimates from Jiang et al., assuming 50 Hz control frequency ($\Delta t = 20$ ms).

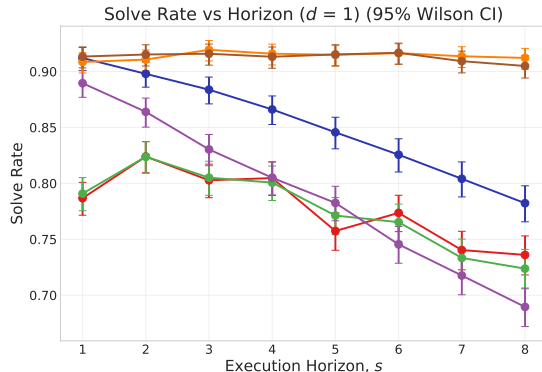
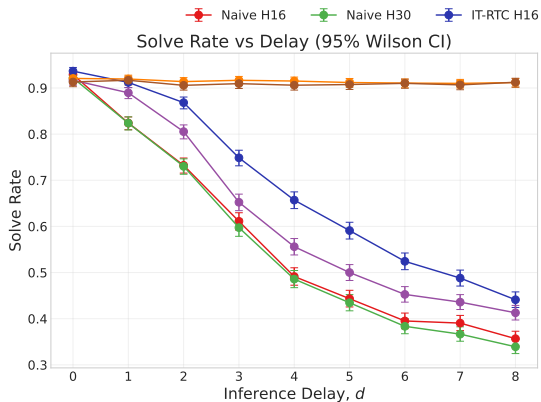


Appendix: Baseline Sanity Check — $d = 0$ Performance



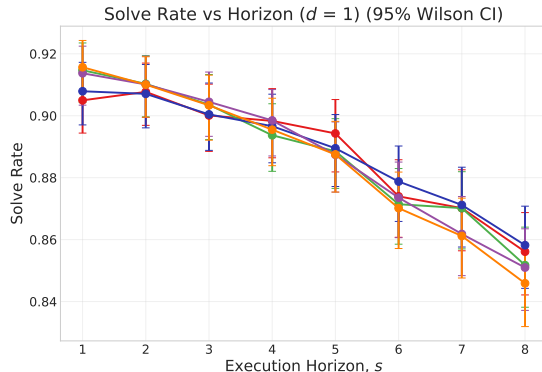
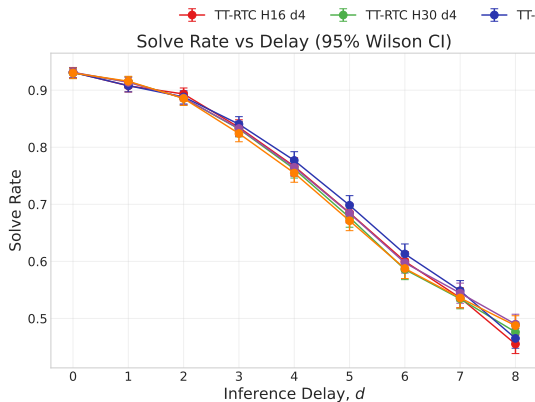
All methods achieve comparable $d = 0$ performance. Minor variations for VLASH with high $d_{\max}$.

Appendix: Cross-Chunk-Size Comparison (Naive, IT-RTC, A2C2)



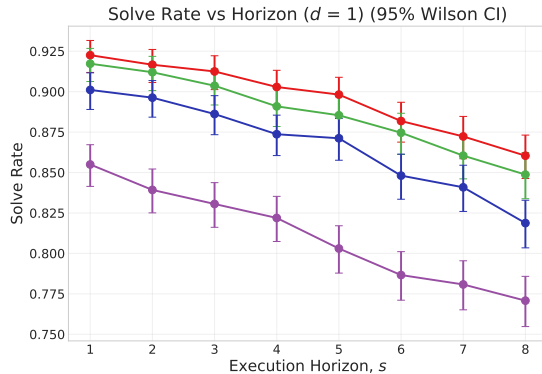
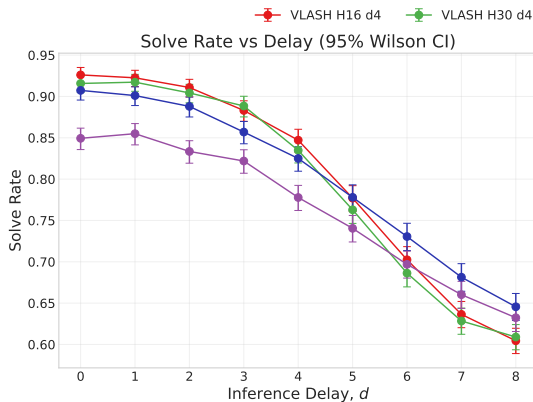
$H = 16$ consistently outperforms $H = 30$ — longer chunks impose intrinsic prediction difficulty.

Appendix: Cross-Chunk-Size Comparison (TT-RTC)



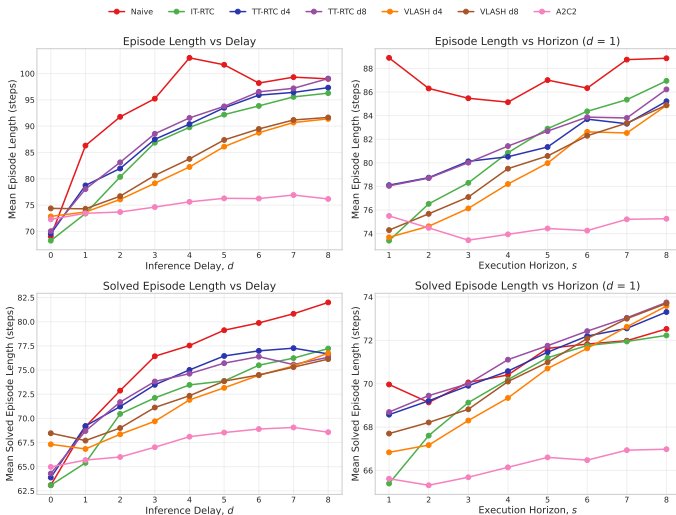
TT-RTC's delay robustness transfers across chunk sizes when $d_{\max}$ is set appropriately.

Appendix: Cross-Chunk-Size Comparison (VLASH)



VLASH's state rollforward accuracy is more sensitive to chunk size than TT-RTC's prefix conditioning.

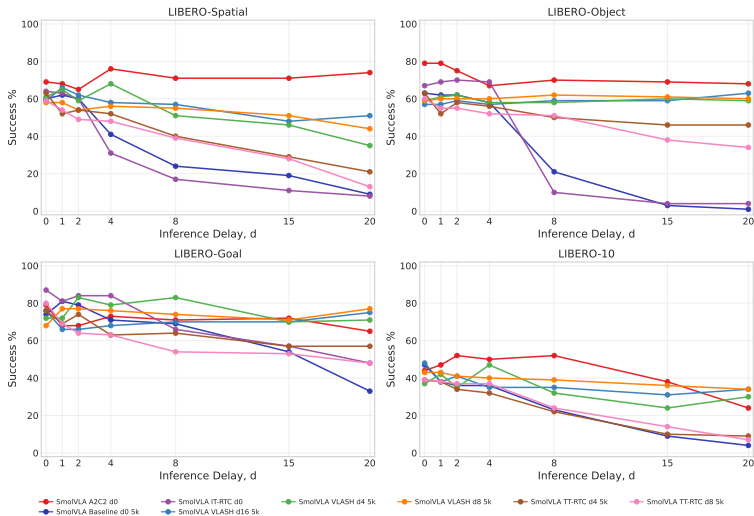
Appendix: Episode Efficiency ($H = 16$)



A2C2 achieves the lowest time-to-solve among solved episodes. Naive requires the most steps.

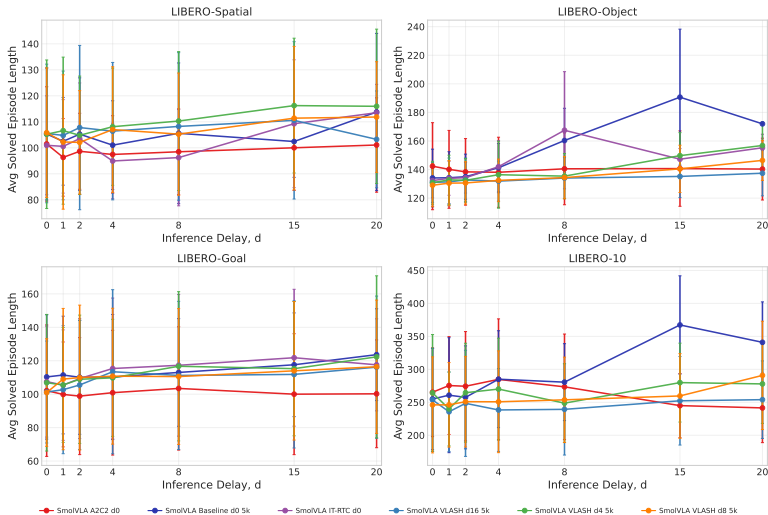
Appendix: LIBERO Per-Suite Success Rate Breakdown

Success % vs Delay (chunk_size=50)

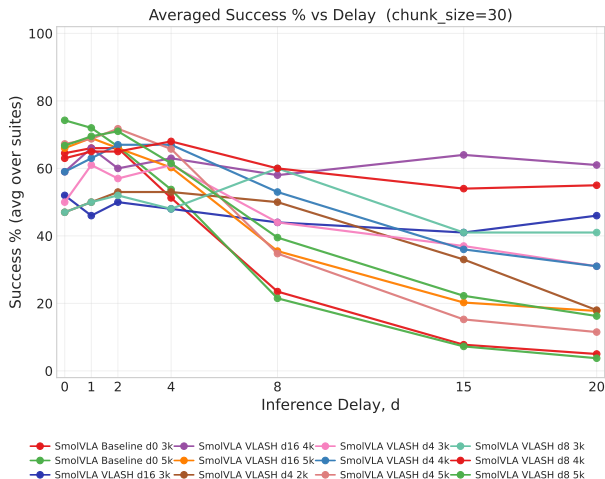


Appendix: LIBERO Per-Suite Episode Length

Avg Solved Episode Length vs Delay (chunk_size=50)



Appendix: VLASH Training Ablation on LIBERO



VLASH variants with $d_{\max} \in \{0, 4, 8, 16\}$ at multiple training checkpoints (2k–5k steps), execution horizon $s = 30$.

Appendix: Kinetix Inference FLOPs per Method

Method	Per-chunk FLOPs	MFLOPs	Ratio
Naive	$N_s \cdot F_{\text{fwd}}$	235	$1.0\times$
IT-RTC	$3N_s \cdot F_{\text{fwd}}$	707	$3.0\times$
TT-RTC	$N_s \cdot F_{\text{fwd}}$	235	$1.0\times$
VLASH	$N_s \cdot F_{\text{fwd}}$ (+ env sim)	235	$1.0\times$
A2C2	$N_s \cdot F_{\text{fwd}} + s \cdot F_{\text{res}}$	$235 + 0.62s$	$\approx 1.02\times$

$C=16$, $N_s=5$ Euler steps, $F_{\text{fwd}} \approx 47.1$ MFLOPs, $F_{\text{res}} \approx 0.62$ MFLOPs.

On Kinetix (no vision encoder), IT-RTC's $3\times$ overhead is a true $3\times$. At VLA scale, the expensive vision prefill amortizes it to only $1.2\times$.

Appendix: SmolVLA FLOPs Breakdown (LIBERO)

Component	FLOPs
SmolVLA prefill (F_{pre})	494.6 GFLOPs
SmolVLA denoising step (F_{ds})	5.64 GFLOPs
SmolVLA full chunk ($F_{\text{pre}} + 10F_{\text{ds}}$)	551.0 GFLOPs
ResidualTransformer forward (F_{res}^L)	8.21 GFLOPs
SmolVLA total parameters	604.9 M
ResidualTransformer (total / trainable)	32.0 M / 20.8 M

Key insight: Vision encoder prefill = **89.8%** of total chunk cost. The denoising loop (where IT-RTC's overhead applies) is only $\sim 10\%$.



Appendix: Measured LIBERO Inference Latency

Method	Mean (ms)	P95 (ms)	Hz	Ratio
Naive	405.2	407.5	2.47	1.00×
TT-RTC	402.7	404.9	2.48	0.99×
IT-RTC	469.7	488.6	2.13	1.16×
A2C2 (serial)	412.4	417.2	2.42	1.02×
A2C2 residual only	7.27	7.35	137.5	0.02×

RTX 3090, SmolVLA, batch size 1, 100 timed iterations

Key takeaways:

- TT-RTC indistinguishable from naive (402.7 vs. 405.2 ms)
- IT-RTC +15.9% latency (2.47 $\rightarrow$ 2.13 Hz) — slowest method
- A2C2 residual head cheap in isolation (7.27 ms), serial path close to naive



Appendix: Break-Even Analysis (TT-RTC vs. IT-RTC)

When does TT-RTC's training investment pay off?

$$N^* = \frac{C_{\text{train}}^{\text{TT-RTC}}}{\Delta C_{\text{chunk}}} \quad \text{where} \quad \Delta C_{\text{chunk}} = C_{\text{chunk}}^{\text{IT-RTC}} - C_{\text{chunk}}^{\text{TT-RTC}}$$

LIBERO (SmoIVLA):

- $C_{\text{train}}^{\text{TT-RTC}} \approx 1,310$ PFLOPs (dominated by vision encoder forward)
- $\Delta C_{\text{chunk}} = 20F_{\text{ds}} = 112.8$ GFLOPs (only denoising saved)
- $N^* \approx 11.6$ M chunks (≈ 2.9 M episodes)

In pure FLOPs: IT-RTC is surprisingly efficient at VLA scale ($1.2\times$ overhead)

In latency: IT-RTC triples the sequential denoising bottleneck $\rightarrow$ **avoid for real-time**



Appendix: Flow Matching for Action Generation

Interpolant: $A^\tau = (1 - \tau)\epsilon + \tau A_0, \quad \epsilon \sim \mathcal{N}(0, I)$

Training loss:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau, \epsilon} [\|v_\theta(A^\tau, o, \tau) - (A_0 - \epsilon)\|^2]$$

Inference (Euler integration from noise):

$$A^{\tau+1/n} = A^\tau + \frac{1}{n} v_\theta(A^\tau, o, \tau)$$

Key properties:

- Deterministic straight-line trajectories (vs. stochastic diffusion)
- Generates full action chunk in parallel (no autoregressive tokens)
- Used by π_0 , $\pi_{0.5}$, SmolVLA, and Kinetix policies



Appendix: Per-Environment Kinetix Breakdown ($H = 16$)

